CSA0613 DESIGN ANALYSIS OF ALGORITHMS

CO2_ASSESSMENT TOOL 2

Q26. An e-commerce system stores order IDs unsorted due to continuous transactions
and uses Linear Search for retrieval.
a) Explain how Linear Search operates in this case.
b) Analyze performance degradation as data grows.
c) Suggest improvements and justify.

Aim

To implement Linear Search for finding an order ID in an unsorted list of e-commerce orders.

Problem Statement

An e-commerce system stores order IDs in an unsorted manner because new transactions occur continuously. Implement a Linear Search algorithm to find a given order ID, analyze its performance as the data size increases, and suggest suitable improvements.

Theory

Linear Search is the simplest searching technique. It checks each element of a list one by one until the required element is found or the list ends. It is suitable for unsorted data but becomes slower as the size of the dataset increases.

Algorithm

1. Read the list of order IDs.
2. Read the order ID to be searched.
3. Start from the first element.
4. Compare each order ID with the search key.
5. If found, display its position.
6. Otherwise, continue until the end of the list.
7. If not found, display “Order ID not found.”

Python Code

```
order_ids = list(map(int, input("Enter order IDs: ").split()))
key = int(input("Enter order ID to search: "))

for i in range(len(order_ids)):
    if order_ids[i] == key:
        print("Order ID found at position", i + 1)
        break
else:
    print("Order ID not found")
```

Output

```
Output
Enter order IDs: 105 120 110 135 140
Enter order ID to search: 135
Order ID found at position 4

=== Code Execution Successful ===
```

Explanation

a) How Linear Search Operates

In an e-commerce system, order IDs are stored in an unsorted list because new orders are continuously added. Linear Search starts from the first order ID and compares each ID with the required order ID until it is found or the list ends. Since the data is unsorted, every element may need to be checked.

b) Performance Degradation as Data Grows

Case	Time Complexity
Best Case	**O(1)**
Average Case	**O(n)**
Worst Case	**O(n)**

As the number of orders increases, the search time increases proportionally because every order ID may have to be checked. This makes Linear Search inefficient for large e-commerce databases.

c) Suggested Improvements

1. Binary Search
    - Sort the order IDs first.
    - Search time becomes O(log n).
2. Hash Table (Dictionary)
    - Average search time is O(1).
    - Suitable for frequent order lookups.
3. Database Indexing
    - Use indexes such as B-Tree.
    - Enables fast retrieval even for millions of records.

Justification:

These techniques reduce search time, improve efficiency, and provide faster response compared to Linear Search in large-scale systems.

Time Complexity

- Best Case: O(1)
- Average Case: O(n)
- Worst Case: O(n)

Space Complexity

O(1)

Only a few extra variables are used regardless of the input size.

Advantages

- Simple to understand and implement.

- Works on both sorted and unsorted data.
- No preprocessing or sorting required.
- Suitable for small datasets.

Disadvantages

- Slow for large datasets.
- Checks every element sequentially.
- Not efficient for frequent searches.
- Poor scalability.

Applications

- Searching order IDs in small e-commerce systems.
- Searching customer records.
- Finding product IDs in unsorted inventories.
- Searching student records or employee IDs.
- Small database applications.

Analysis

Linear Search is suitable for unsorted data because it does not require sorting before searching. However, its O(n) time complexity causes performance to degrade as the number of order IDs increases. In large e-commerce applications, techniques such as

Binary Search, Hash Tables, or Database Indexing provide much faster retrieval and better scalability.

Result

The Linear Search program was successfully implemented to search an order ID in an unsorted list. The program correctly displayed the position of the order ID when found; otherwise, it displayed “Order ID not found.” The performance analysis showed that Linear Search is efficient only for small datasets, while Binary Search, Hash Tables, and Database Indexing are more suitable for large-scale e-commerce applications.